Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Juego Web de Naves Espaciales (PROYEC-21)

1. CONTEXTO
Se requiere un juego arcade estilo años 80 completamente en HTML5, CSS y JavaScript puro. El requerimiento especifica explícitamente la estructura de carpetas con archivos JS separados (game.js, controls.js, obstacles.js). El entorno de ejecución es un archivo estático que puede abrirse directamente en el navegador (protocolo file:///) o servirse desde un servidor HTTP simple.

2. EVALUACIÓN DE ENTORNO CRÍTICA
El requerimiento indica 'HTML5, CSS y JS en su totalidad' sin mencionar bundlers (Webpack, Vite), frameworks (React, Vue) ni servidor Node. Esto implica un entorno FILE:// o servidor HTTP estático simple. DECISIÓN: Se prohíbe el uso de `<script type='module'>` e `import/export` nativos de ES6 para garantizar compatibilidad máxima sin servidor. Todos los scripts JS se cargarán como scripts clásicos en orden secuencial en el index.html, compartiendo el scope global controlado mediante un namespace único `GalaxyGame`. Esto resuelve la modularidad sin romper la compatibilidad con file:///.

3. DECISIÓN ARQUITECTÓNICA: Monolito Frontend Estático con Namespace Global
- **Patrón elegido**: Single-File Architecture expandida con namespace global (`window.GalaxyGame = window.GalaxyGame || {}`). Cada archivo JS declara su módulo dentro del namespace sin usar import/export.
- **Motor de juego**: Canvas API nativa del navegador (HTML5 Canvas 2D). Sin librerías externas (Phaser, PixiJS, Three.js). YAGNI: el juego 2D arcade no requiere un motor completo.
- **Renderizado**: Game Loop clásico con `requestAnimationFrame`. Delta time para movimiento independiente del framerate.
- **Assets gráficos**: 100% generados por código (Canvas API con formas geométricas, gradientes y efectos de píxeles). CERO archivos de imagen externos. Esto garantiza que el juego funcione offline y sin servidor. El estilo arcade 80s se logra con fuentes monoespaciadas, colores neón sobre fondo negro y formas vectoriales simples.
- **Audio**: Web Audio API nativa para efectos de sonido procedurales (beeps, explosiones). Sin archivos .mp3/.wav externos.

4. JUSTIFICACIÓN YAGNI / KISS
- **¿Por qué NO Phaser.js?** El requerimiento pide HTML5/CSS/JS puro. Phaser añade 1MB+ de dependencia externa, requiere CDN o npm, y oculta la lógica del Canvas que los criterios de aceptación implícitamente esperan ver. YAGNI: Canvas API resuelve el 100% del caso de uso.
- **¿Por qué NO módulos ES6 nativos?** Sin servidor HTTP confirmado, `type='module'` falla en file:///. El namespace global es la solución probada y compatible.
- **¿Por qué NO archivos de imagen en assets/?** La restricción del skill prohíbe binarios. Los gráficos procedurales con Canvas son superiores para el estilo arcade 80s (píxeles, vectores, efectos de scanline).
- **¿Por qué NO localStorage para high scores?** Se incluye porque agrega valor real con cero dependencias externas.
- **¿Por qué NO responsive.css separado?** El juego usa Canvas con dimensiones fijas (800x600). El responsive se maneja con CSS transform:scale() en un único styles.css. Consolidación justificada.

5. STACK TECNOLÓGICO (máximo 2 componentes externos)
- **Componente 1**: Navegador moderno (Chrome/Firefox/Edge) — Canvas 2D API + Web Audio API + requestAnimationFrame

- **Componente 2**: Google Fonts CDN (fuente 'Press Start 2P' para el look arcade 80s) — única dependencia externa, justificada estéticamente
- **Componente 3 (opcional/fallback)**: Si no hay internet, CSS font-family: monospace como fallback automático

6. ARQUITECTURA DE MÓDULOS (Namespace GalaxyGame)
- `config.js`: Constantes del juego (velocidades, niveles, colores neón, dimensiones)
- `utils.js`: Funciones matemáticas (colisiones AABB, random, lerp, vector2D)
- `renderer.js`: Funciones de dibujo Canvas (nave, obstáculos, estrellas, efectos, HUD)
- `entities.js`: Clases/constructores para Nave, Obstáculo, PowerUp, Partícula
- `controls.js`: Gestión de teclado (keydown/keyup, estado de teclas)
- `obstacles.js`: Factory de obstáculos (cometas, asteroides, planetas, OVNIs, meteoritos) y power-ups
- `audio.js`: Web Audio API procedural (efectos de sonido sintéticos)
- `game.js`: Game loop principal, gestión de estados (menu/playing/gameover), niveles, puntuación

7. SISTEMA DE JUEGO
- **5 tipos de obstáculos**: Cometa (rápido, diagonal), Asteroide (mediano, errático), Planeta (grande, lento), OVNI (inteligente, persigue), Meteorito (pequeño, enjambre)
- **3 tipos de power-ups**: Escudo (invulnerabilidad temporal), Turbo (velocidad máxima), Imán (atrae puntos)
- **3 niveles de dificultad**: Nebulosa (fácil), Galaxia (medio), Agujero Negro (difícil) — velocidad y densidad de obstáculos escalan
- **Puntuación**: Tiempo de supervivencia (pts/segundo) + bonus por power-ups recolectados
- **Controles**: !• !’ para girar la nave, !‘ para acelerar, !“ para frenar (velocidad mínima > 0)

8. RIESGOS ASUMIDOS POR SIMPLICIDAD
- Sin persistencia de partidas (solo high score en localStorage)
- Sin sonido en Safari iOS (restricción de Web Audio API sin interacción previa — mitigado con botón de inicio)
- Sin multijugador (fuera de scope del requerimiento)
- Gráficos procedurales pueden no satisfacer expectativas visuales muy altas — mitigado con efectos de partículas y animaciones CSS

2. MÉTRICAS DE ITERACIÓN (QA)

- js/config.js | Estado: APROBADO | Iteraciones: 1
- index.html | Estado: APROBADO | Iteraciones: 1
- js/utils.js | Estado: APROBADO | Iteraciones: 1
- js/audio.js | Estado: APROBADO | Iteraciones: 1
- js/renderer.js | Estado: APROBADO (post-integration-fix) | Iteraciones: 1
- js/entities.js | Estado: APROBADO (post-integration-fix) | Iteraciones: 1
- js/controls.js | Estado: APROBADO | Iteraciones: 1
- js/obstacles.js | Estado: APROBADO (post-integration-fix) | Iteraciones: 1
- js/game.js | Estado: APROBADO (post-integration-fix) | Iteraciones: 1
- css/styles.css | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 446,762
Tokens de salida (output): 92,471
Total tokens: 539,233
Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~56.3 horas-hombre ahorradas
"H squad de 4 personas × 7.1 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“2,517 input | !‘2,188 output | 1 llamadas | ~1.8h ahorradas
MICROPLANIFICACION-DISEÑO: !“49,156 input | !‘26,199 output | 11 llamadas | ~19.5h ahorradas
FRONTEND-DESARROLLO: !“198,782 input | !‘46,959 output | 14 llamadas | ~18.5h ahorradas

QA-VALIDACION: !“196,307 input | !‘17,125 output | 16 llamadas | ~16.6h ahorradas